



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sistemas Embebidos

Departamento de Ingeniería Eléctrica y Electrónica

Sistema embebido inalámbrico para el reconocimiento de gestos

Proyecto final de curso

Estudiantes:

Julián Camilo Casallas Villamil
Miguel Angel Cleves Bolivar
Salomón Velasco Rueda

Docente:

Andrés Junior Gallego Garcés

Fecha:

1 de agosto de 2026

Período Académico 2026-I

Índice general

1	Resumen	3
2	Introducción	3
2.1	Contexto y planteamiento del problema	3
2.2	Justificación	4
2.3	Objetivos	4
2.3.1	Objetivo general	4
2.3.2	Objetivos específicos	4
2.4	Metodología de desarrollo y validación	5
3	Alcance y requisitos del sistema	5
3.1	Alcance funcional	5
3.2	Requisitos y estado comprobado	6
3.3	Restricciones	6
4	Arquitectura del sistema	7
4.1	Arquitectura general	7
4.2	Secuencia de operación de v4	7
4.3	Contrato de datos	8
5	Implementación de hardware	8
5.1	Componentes	8
5.2	Montaje experimental	9
5.3	Asignación de señales	10
5.4	Consideraciones eléctricas	11
6	Diseño y evolución del firmware ESP32	11
6.1	Adquisición e integridad de las muestras	11
6.2	Validación de movimiento	11
6.3	Pulsador, comunicación y respuesta	12
6.4	OLED, servo y LED	12
6.5	Comparación de versiones	13
6.6	Adaptación del APDS-9960	13
7	Conjunto de datos e inteligencia artificial	14
7.1	Adquisición y comprobación del conjunto real	14
7.2	División y normalización	15
7.3	Arquitectura de la CNN	16
7.4	Entrenamiento y evaluación	16
7.5	Exportación e inferencia NumPy	18

8	Software de la Raspberry Pi e interfaz IoT	18
8.1	Servidor Flask	18
8.2	Prueba del dashboard y el monitor serial	19
8.3	Arranque, red y seguridad	20
9	Diseño de la PCB	20
9.1	Alcance del diseño	20
9.2	Correspondencia y revisión previa a fabricación	20
10	Resultados y discusión	23
10.1	Resultados principales	23
10.2	Fallo de cierre: <i>up</i> clasificado como <i>left</i>	23
10.3	Problemas encontrados y medidas implementadas	24
10.4	Estado final del proyecto	25
11	Limitaciones y trabajo pendiente	25
12	Conclusiones	26
A	Trazabilidad de los archivos principales	27



1 Resumen

Este informe documenta el diseño, la implementación y la evaluación de un sistema embebido inalámbrico para reconocer los gestos *left*, *right* y *up*. El prototipo distribuye las tareas entre un ESP32, encargado de adquirir señales de un MPU6050 y controlar la interfaz local, y una Raspberry Pi Zero W, destinada a recibir las capturas mediante HTTP y ejecutar una red neuronal convolucional unidimensional. Cada observación contiene 100 muestras de seis canales inerciales, tomadas nominalmente a 50 Hz durante aproximadamente dos segundos. El APDS-9960 se utilizó como referencia para asignar etiquetas durante la construcción del conjunto de datos, pero no forma parte de la entrada del modelo final.

Se verificó un conjunto balanceado de 120 capturas reales, 40 por clase, sin archivos finales inválidos, secuencias completamente nulas, secuencias congeladas ni duplicados exactos. La CNN de 2643 parámetros alcanzó una exactitud de prueba de 83,33 % sobre 24 observaciones: 8/8 aciertos para *left*, 6/8 para *right* y 6/8 para *up*. La equivalencia numérica entre la evaluación Keras y la reconstrucción de la inferencia con NumPy presentó una diferencia máxima de $1,788 \times 10^{-7}$.

La versión v4 incorpora operación I2C a 50 kHz, recuperación del bus, rechazo de muestras nulas o congeladas, un criterio de movimiento más exigente, control robustecido del pulsador, pantalla OLED SPI, servomotor y LED. El prototipo se probó energizado y, durante una de las ejecuciones, la OLED mostró la clase *RIGHT* con 49,42 % de confianza. También se completó una transacción entre el ESP32 y el servidor Flask con respuesta *left*; no obstante, en esa prueba no quedó registrada de manera inequívoca la versión del modelo que se encontraba activa.

El servicio `gesture-server.service`, los endpoints Flask y la inferencia con NumPy se comprobaron en funcionamiento durante el desarrollo. Sin embargo, el paquete local del modelo real no contiene el módulo de inferencia NumPy compatible necesario para reconstruir directamente el despliegue, y no quedó registrado de manera concluyente qué versión del modelo estaba activa en una prueba final que produjo *up* → *left*. Además, la PCB fue diseñada, pero no fabricada ni validada, y se identificaron conexiones que deben corregirse antes de producirla. Estas limitaciones permiten distinguir con claridad los resultados obtenidos de las mejoras que aún están pendientes.

Palabras clave. Sistemas embebidos, ESP32, Raspberry Pi Zero W, MPU6050, reconocimiento de gestos, CNN 1D, NumPy, Flask, IoT.

2 Introducción

2.1 Contexto y planteamiento del problema

Un sistema de reconocimiento gestual basado en señales inerciales reúne problemas propios de la ingeniería embebida: adquisición periódica, respuesta a eventos, integridad del bus de sensores, comunicación inalámbrica, procesamiento de datos, inferencia y actuación. En este proyecto se buscó reconocer tres movimientos de la mano con componentes de bajo costo y separar la operación entre una plataforma de control y una plataforma Linux. El ESP32 ofrece interfaces digitales, temporización y Wi-Fi en un solo dispositivo [1]; la Raspberry Pi Zero W proporciona un entorno apropiado para un servicio web liviano [4].



El problema técnico no se reduce a obtener una etiqueta de una red neuronal. Para que el resultado sea útil, la captura debe contener movimiento real, mantener siempre el mismo orden temporal y de canales, llegar íntegra al servidor y normalizarse con los parámetros del entrenamiento. Asimismo, la versión exacta del modelo, el orden de clases y el código de inferencia deben permanecer sincronizados. Las fallas observadas durante el desarrollo —lecturas en cero, secuencias congeladas, eventos duplicados de pulsador y una clasificación final *up*→*left*— muestran que la calidad del sistema depende de toda la cadena y no únicamente de la métrica del modelo.

2.2 Justificación

La arquitectura propuesta permite estudiar en una sola aplicación sensores, actuadores, firmware, redes, servicios en Linux e inteligencia artificial. El microcontrolador mantiene las tareas cercanas al hardware, mientras la Raspberry Pi concentra la aplicación Flask y el cálculo de la CNN. Esta división facilita observar el sistema, actualizar el modelo sin recompilar el firmware y comparar el comportamiento fuera de línea con las respuestas recibidas durante una demostración. El proyecto también permite discutir trazabilidad: una exactitud de prueba no demuestra por sí sola que el mismo artefacto esté cargado en el servidor.

2.3 Objetivos

2.3.1 Objetivo general

Diseñar, implementar y evaluar un sistema embebido inalámbrico capaz de clasificar tres gestos a partir de señales del MPU6050, mediante la integración de un ESP32, una Raspberry Pi Zero W, una CNN 1D, un servicio Flask y una interfaz local con pantalla y actuadores.

2.3.2 Objetivos específicos

- Adquirir secuencias de 100 muestras de aceleración y velocidad angular a una frecuencia nominal de 50 Hz después de una solicitud por pulsador.
- Construir y comprobar un conjunto balanceado de capturas reales para *left*, *right* y *up*, usando el APDS-9960 como referencia de etiquetado.
- Entrenar y evaluar una CNN 1D compacta, sin mezclar observaciones entre los conjuntos de entrenamiento, validación y prueba.
- Verificar que una implementación manual con NumPy reproduzca la salida del modelo Keras.
- Integrar la transferencia de capturas y resultados mediante Wi-Fi, HTTP y JSON.
- Mostrar el estado y la inferencia en una OLED, y asociar las clases con posiciones del servomotor y patrones del LED.
- Robustecer el firmware frente a rebote del pulsador, fallos del bus I2C, lecturas nulas, muestras congeladas y capturas con movimiento insuficiente.
- Documentar el diseño de PCB sin confundirlo con una tarjeta fabricada o eléctricamente validada.



2.4 Metodología de desarrollo y validación

El desarrollo se documentó a partir del código fuente .ino y .py, los 120 archivos JSON, el cuaderno de entrenamiento, los paquetes del modelo, los manifiestos, los respaldos, las pruebas realizadas y los archivos de KiCad. Para presentar el estado de cada parte del sistema se emplean cuatro categorías:

- **Comprobado:** el comportamiento o valor se validó mediante código, datos o una prueba funcional.
- **Probado de forma puntual:** se obtuvo una salida durante una ejecución, pero no se realizó una campaña completa de ensayos.
- **Implementado sin validación completa:** existe código o diseño, pero aún falta una prueba reproducible del conjunto.
- **Pendiente:** se identificó una tarea o contradicción que todavía debe resolverse.

3 Alcance y requisitos del sistema

3.1 Alcance funcional

El flujo previsto comienza con una pulsación. El ESP32 verifica el evento, presenta una secuencia de preparación, lee el MPU6050, valida la integridad y el movimiento de la captura, construye el JSON y lo envía a la Raspberry Pi. El servidor debe normalizar la matriz de entrada, ejecutar la CNN y devolver clase, confianza y probabilidades. Finalmente, el ESP32 actualiza la OLED, genera el patrón del LED y acciona el servo. El APDS-9960 pertenece al proceso de creación del conjunto de datos; en v4 se encuentra deshabilitado lógicamente y no interviene en la inferencia.

La PCB y el buzzer se incluyen con un alcance distinto. La primera es una propuesta de integración física elaborada en KiCad y el segundo se probó de manera independiente, pero el firmware v4 no implementa su control. En consecuencia, ninguno forma parte de la integración funcional final.

3.2 Requisitos y estado comprobado

Requisito	Estado	Resultado o restricción
Captura MPU6050 de 100×6	Implementado y verificado en código	Las versiones v2-v4 reservan 100 muestras y seis canales; los 120 JSON presentan esa forma.
Activación mediante pulsador	Implementado	GPIO27 con INPUT_PULLUP, interrupción de flanco y validación posterior fuera de la ISR.
Transferencia HTTP/JSON	Funcionamiento comprobado	Se realizaron solicitudes de 100 muestras con respuesta HTTP 200; en la prueba registrada no quedó identificada la versión exacta del modelo activo.
Clasificación CNN de tres clases	Verificada fuera de línea	Artefactos del modelo real, métricas, historial y manifiesto coherentes; exactitud de prueba de 83,33 %.
Inferencia NumPy en Raspberry	Implementada y probada	La inferencia se ejecutó en la Raspberry; el paquete local del modelo real no incluye el módulo de ejecución compatible.
Interfaz OLED	Implementada y probada	Durante una ejecución mostró RIGHT y 49,42 %; no se realizó una campaña para medir su desempeño por clase.
Respuesta del servo y LED	Implementada en v4	Ambos actuadores se integraron al prototipo; falta una caracterización sistemática de su respuesta para las tres clases.
Arranque automático con systemd	Configurado y comprobado	El servicio <code>gesture-server.service</code> se verificó activo y con reinicio automático ante fallos.
PCB de integración	Diseñada, no fabricada	Se elaboraron el esquemático y el ruteo; no se generaron Gerbers ni se realizaron ERC/DRC, montaje o pruebas eléctricas.

Tabla 1: Estado de los requisitos principales del sistema.

3.3 Restricciones

La Raspberry Pi Zero W dispone de un procesador de una sola CPU ARM11 a 1 GHz y 512 MB de memoria, además de Wi-Fi de 2,4 GHz [4]. En esta plataforma no fue posible utilizar de manera apropiada TensorFlow ni `tflite_runtime`; por ello se desarrolló una ruta de inferencia con NumPy. La red de demostración se apoyó en un punto de acceso portátil y una dirección fija configurada para el servidor, por lo que la disponibilidad dependía de ese entorno. En el lado eléctrico, el SG90 requiere 5 V y tierra común; alimentarlo desde 3,3 V o sin reserva de corriente puede inducir caídas de tensión y reinicios.

4 Arquitectura del sistema

4.1 Arquitectura general

La Figura 1 distingue el flujo de inferencia del flujo histórico de etiquetado. La entrada efectiva de la CNN es exclusivamente el arreglo inercial del MPU6050.

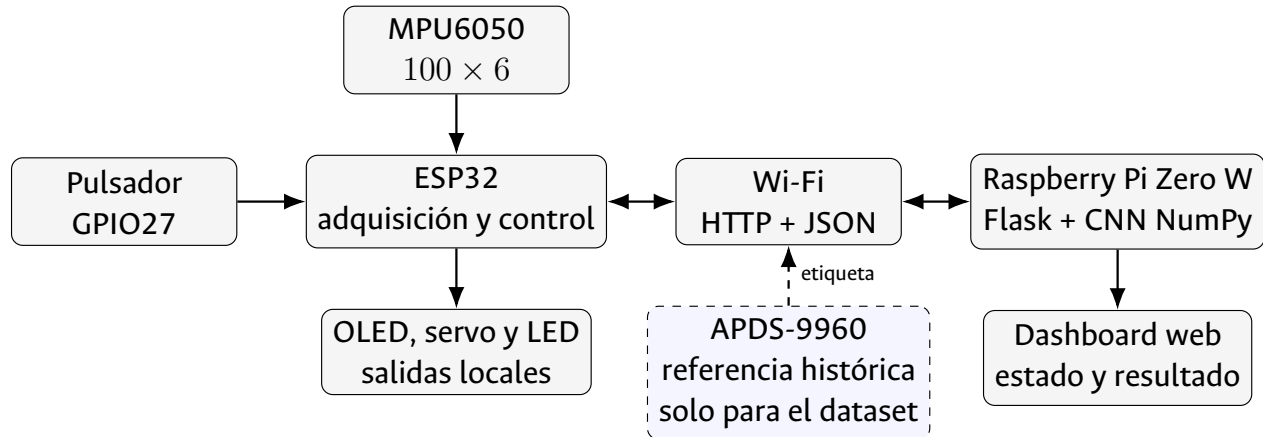


Figura 1: Arquitectura distribuida y separación entre la inferencia final y el etiquetado histórico con APDS-9960.

4.2 Secuencia de operación de v4

1. Se inicializan la OLED por SPI, el MPU6050 sobre I2C, la conexión Wi-Fi, el servo, el LED y el pulsador. El APDS-9960 permanece deshabilitado por configuración.
2. Una interrupción de flanco descendente registra la solicitud; el procesamiento se realiza fuera de la rutina de interrupción.
3. El nivel bajo del pulsador debe persistir 30 ms antes de aceptar el evento. Después del ciclo, la liberación debe permanecer estable 80 ms.
4. El firmware lee 100 muestras del MPU6050, con periodo nominal de 20 ms. No se actualizan la OLED ni el servo dentro del lazo.
5. Se rechazan datos físicamente débiles, completamente nulos, repetidos de manera anómala o sin variación suficiente.
6. La captura aceptada se serializa y se envía a `/api/gesture`.
7. La respuesta del servidor se interpreta para presentar clase, confianza y estado. No existe en v4 un umbral mínimo de confianza antes de actuar.
8. El servo se adjunta para el movimiento, permanece activo 300 ms y se desacopla; el LED emite un número de pulsos dependiente de la clase.

4.3 Contrato de datos

Cada muestra conserva el orden ax, ay, az, gx, gy, gz . La consistencia de este orden es tan importante como el número de muestras, pues una permutación entre entrenamiento y despliegue cambia el significado físico de cada entrada. El servidor espera una matriz de forma $(100, 6)$ y la salida Softmax se interpreta en el orden *left, right, up*, según la configuración del modelo real.

5 Implementación de hardware

5.1 Componentes

Elemento	Función	Observación
ESP32 DevKit V1	Cliente de adquisición y control	Placa de desarrollo basada en ESP32-WROOM-32; usa Wi-Fi, interrupción, I2C, SPI y PWM.
Raspberry Pi Zero W v1.1	Servidor de aplicación	Ejecuta Linux, Flask y la ruta de inferencia; el servicio y los endpoints se comprobaron durante el desarrollo.
MPU6050	Sensor de movimiento	Acelerómetro y giróscopo triaxiales, dirección I2C 0x68 [2].
APDS-9960	Referencia de etiqueta	Reconocimiento óptico de gestos durante la adquisición; deshabilitado en v4 y ajeno a la entrada de la CNN [3].
OLED SSD1306	Interfaz local	Pantalla monocromática de 128×64 ; usa MOSI/CLK y las señales DC/CS/RESET.
Servo SG90	Actuador mecánico	Posiciones nominales de 0° , 90° y 180° ; alimentación prevista de 5 V.
LED y pulsador	Salida y entrada discretas	LED de estado en GPIO26 y pulsador activo en bajo en GPIO27.
Buzzer activo	Prueba o expansión auxiliar	Existe un ensayo independiente y aparece en la PCB, pero no es controlado por v4.

Tabla 2: Componentes principales y función dentro del alcance documentado.

La Figura 2 presenta la placa utilizada durante las pruebas, alimentada mediante su conexión micro-USB.



Figura 2: Placa de desarrollo basada en ESP32, alimentada mediante conexión micro-USB durante las pruebas del prototipo.

5.2 Montaje experimental

El prototipo se ensambló sobre dos protoboards para permitir cambios rápidos de cableado y pruebas independientes de periféricos. En la Figura 3 se muestra el montaje energizado con la OLED, el servo, el pulsador, los indicadores, las resistencias, los módulos y sus conexiones.

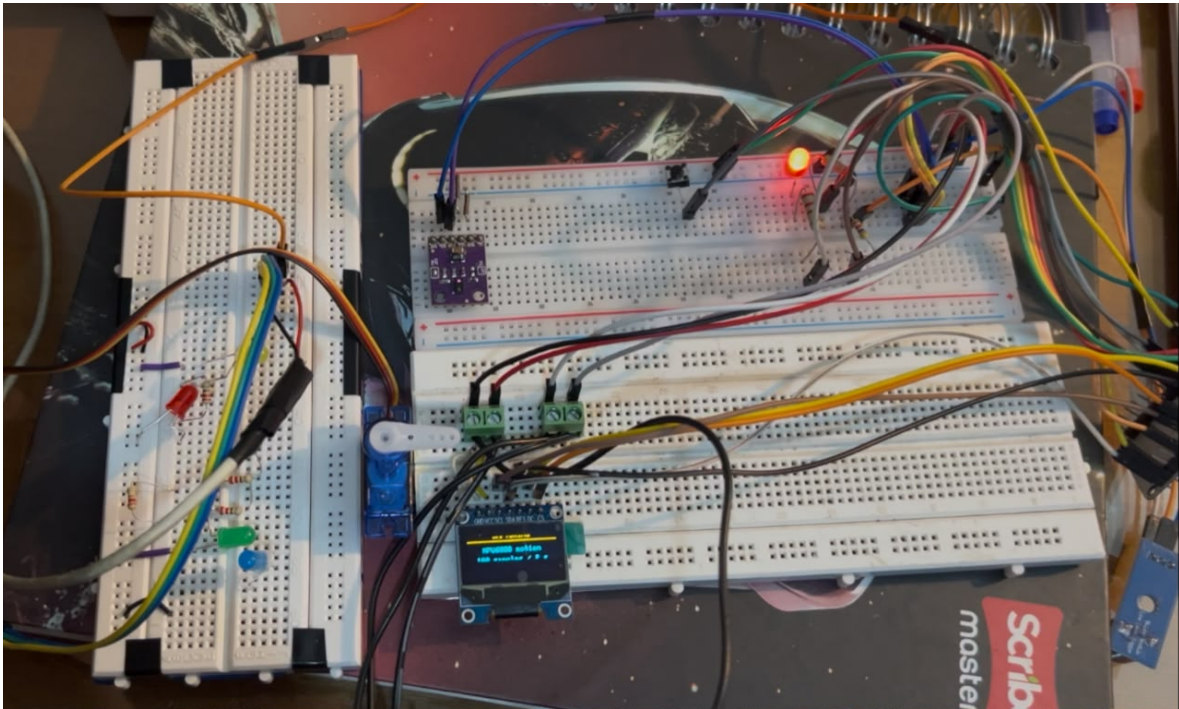


Figura 3: Prototipo experimental del sistema de reconocimiento de gestos montado sobre protoboard. Se observan la pantalla OLED, el microservo, indicadores LED, el pulsador, módulos auxiliares y el cableado de interconexión durante una prueba energizada.

5.3 Asignación de señales

La Tabla 3 se basa en las constantes de v4. Las líneas del APDS indican la asignación compatible con el bus compartido del firmware, no la conexión errónea encontrada en el esquemático de la PCB.

Señal	ESP32	Uso en v4
MPU6050 SDA / SCL	GPIO21 / GPIO22	Bus I2C activo a 50 kHz.
APDS-9960 SDA / SCL	GPIO21 / GPIO22	Asignación prevista; periférico deshabilitado lógicamente.
Pulsador	GPIO27	INPUT_PULLUP, interrupción de flanco descendente.
LED de estado	GPIO26	Uno, dos o tres pulsos para <i>left</i> , <i>up</i> y <i>right</i> ; seis para error o clase desconocida.
OLED MOSI / CLK	GPIO23 / GPIO18	Datos y reloj SPI.
OLED DC / CS / RESET	GPIO2 / GPIO5 / GPIO4	Control de la OLED.
Servo	GPIO13	Señal PWM; adjunto únicamente durante la actuación.
Buzzer	GPIO25	Disponible en prueba independiente; no aparece en la lógica de v4.

Tabla 3: Asignación de señales de v4 y de la prueba auxiliar del buzzer.

5.4 Consideraciones eléctricas

El ESP32-WROOM-32 opera con alimentación nominal de 3,3 V y el módulo especifica un intervalo de 3,0 a 3,6 V [1]. El servo debe conectarse a una fuente de 5 V con tierra común y capacidad suficiente para los picos de corriente. Conviene ubicar desacoplo local cerca del servo y evitar que su retorno de corriente atraviese la referencia sensible del sensor. Los módulos I2C deben compartir niveles compatibles y disponer de resistencias de polarización; el esquemático de la PCB no muestra *pull-ups* propios, por lo que su presencia en los módulos debe comprobarse antes de fabricar.

6 Diseño y evolución del firmware ESP32

6.1 Adquisición e integridad de las muestras

Cada captura se representa por

$$\mathbf{x}[n] = [a_x[n] \ a_y[n] \ a_z[n] \ g_x[n] \ g_y[n] \ g_z[n]], \quad n = 0, \dots, 99. \quad (1)$$

El periodo nominal es $T_s = 20$ ms y, por tanto, la frecuencia objetivo es $f_s = 1/T_s = 50$ Hz. La interfaz y los actuadores se actualizan fuera del lazo para reducir la perturbación temporal.

En v4 se comprobó primero la identidad del MPU6050. Ante fallos, el firmware puede generar hasta 16 pulsos sobre SCL, emitir una condición de parada y reiniciar el dispositivo hasta tres veces. Además, descarta una lectura cuando la suma de valores absolutos de aceleración es inferior a 2000 unidades crudas. Si aparecen cinco muestras plausibles idénticas consecutivas —cuatro transiciones sin cambio— se interpreta una posible congelación y se intenta recuperar el bus.

6.2 Validación de movimiento

Para cada grupo de canales se calcula el máximo rango pico a pico:

$$A_{pp} = \max_{c \in \{a_x, a_y, a_z\}} \left(\max_n c[n] - \min_n c[n] \right), \quad (2)$$

$$G_{pp} = \max_{c \in \{g_x, g_y, g_z\}} \left(\max_n c[n] - \min_n c[n] \right). \quad (3)$$

La condición implementada en v4 es

$$A_{pp} \geq 2500 \quad \wedge \quad G_{pp} \geq 2500. \quad (4)$$

La conjunción de ambos criterios es más restrictiva que las versiones previas. El código no implementa límites superiores de saturación ni una prueba física completa de plausibilidad; por ello debe hablarse de filtros específicos y no de una validación universal de la señal.



6.3 Pulsador, comunicación y respuesta

La rutina de interrupción solo registra la solicitud. El filtrado temporal y la espera de liberación se ejecutan en el flujo principal. Esta separación evita hacer operaciones de red, pantalla o adquisición dentro de la ISR. La carga útil utiliza el orden de la Ecuación 1; de forma abreviada:

La espera de liberación comprueba el nivel bajo mediante un ciclo sin tiempo límite. Por tanto, un pulsador atascado puede bloquear indefinidamente el flujo principal; esta protección quedó pendiente.

```
{
  "samples": [
    {"ax": ..., "ay": ..., "az": ...,
     "gx": ..., "gy": ..., "gz": ...}
  ]
}
```

El firmware envía las capturas mediante el método POST al endpoint `/api/gesture`. El mensaje de respuesta incluye los siguientes campos principales:

```
success, gesture, confidence, probabilities
received_samples, model_engine, inference_time_ms
```

La confianza es la probabilidad Softmax de la clase seleccionada y no constituye una garantía de corrección.

6.4 OLED, servo y LED

La OLED representa estados de arranque, conexión, preparación, captura, validación, envío, procesamiento, resultado y error. La Figura 4 muestra una ejecución en la que se obtuvo la clase *RIGHT* con 49,42 % de confianza. Esta prueba permitió comprobar la presentación local del resultado; la respuesta conjunta del gesto, la pantalla y el servo se analiza por separado.



Figura 4: Visualización local de una inferencia en la pantalla OLED: clase RIGHT y confianza de 49,42 %. El microservo se observa junto al módulo de visualización.

Clase	Ángulo nominal	Pulsos del LED
right	0°	3
up	90°	2
left	180°	1
Error o desconocida	Sin mapeo de clase	6

Tabla 4: Mapeo implementado para el servo y el LED en v4.

El servo se desacopla después de 300 ms para reducir interferencia y consumo sostenido. No existe una condición de confianza mínima antes del movimiento y una clase desconocida puede coexistir con una respuesta marcada como exitosa por el servidor. Estas dos situaciones deben corregirse para que la actuación tenga un comportamiento seguro.

6.5 Comparación de versiones

Las versiones se compararon a partir de sus cambios funcionales. El archivo v1 conservado contiene texto ajeno al código fuente y duplicaciones de v2, por lo que no puede compilarse en su estado actual; su alcance funcional se tomó del archivo README.

6.6 Adaptación del APDS-9960

La biblioteca SparkFun se adaptó para aceptar el identificador 0xA8, además de los valores admitidos originalmente. Este cambio permitió trabajar con la variante disponible durante la adquisición. Se trata de una modificación de compatibilidad utilizada durante el etiquetado; v4 define `ENABLE_APDS_REFERENCE=false` y no depende del APDS para inferir.



Versión	Estado del archivo	Cambios principales
v1	Código fuente incompleto	El README describe la integración de OLED y servo, pero el .ino contiene material duplicado y no puede compilarse en su estado actual.
v2	Fuente coherente	I2C a 100 kHz, APDS activo, bloqueo de interrupción durante el ciclo, liberación estable de 80 ms, hasta cuatro intentos de lectura, recuperación tras el segundo fallo y criterio $A_{pp} \geq 1500$ o $G_{pp} \geq 500$. El servo permanece adjunto.
v3	Fuente coherente	APDS deshabilitado, tiempo límite I2C de 50 ms, prueba previa WHO_AM_I, pulsos de recuperación y reinicios del MPU; umbral de giroscopio elevado a 1200, todavía mediante una condición OR. El servo se adjunta solo para actuar.
v4	Fuente coherente, versión final	I2C a 50 kHz, rechazo de lecturas nulas o de baja magnitud, detección de secuencias congeladas, hasta 16 pulsos de recuperación, criterio 2500 AND 2500 y control reforzado del pulsador. Conserva OLED, servo y LED; APDS y buzzer no intervienen.

Tabla 5: Evolución comprobada del firmware integrado hasta v4.

7 Conjunto de datos e inteligencia artificial

7.1 Adquisición y comprobación del conjunto real

El APDS-9960 produjo la etiqueta de referencia mientras el MPU6050 registró la secuencia que posteriormente alimentaría la CNN. La dirección *up* se definió como un movimiento vertical ascendente en el protocolo de captura. El conjunto final contiene 120 JSON, 40 por clase, y cada archivo almacena 100 muestras con los seis canales en representación *raw_int16*. La validación automática confirmó que no existen etiquetas incoherentes, formas inválidas, secuencias enteramente nulas, secuencias congeladas ni duplicados exactos.

La distribución balanceada se muestra en la Figura 5. La gráfica fue generada directamente a partir de los 120 archivos y puede reproducirse con el script incluido en los recursos del informe.

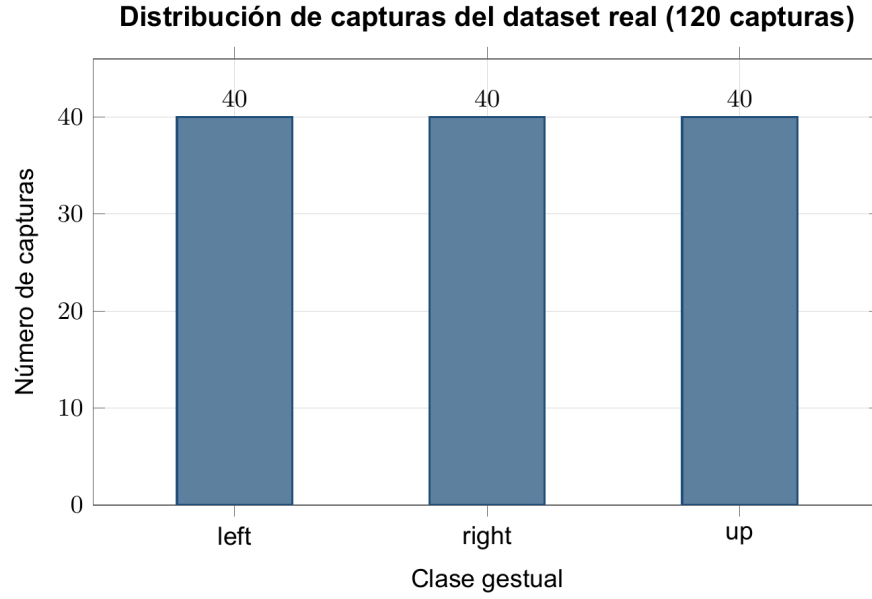


Figura 5: Distribución de las 120 capturas reales por clase, calculada desde los archivos JSON del conjunto final.

Parámetro	Valor verificado
Clases	<i>left, right, up</i>
Capturas	120, 40 por clase
Muestras por captura	100
Canales	6
Frecuencia nominal	50 Hz
Duración nominal	2 s
Representación	raw_int16

Tabla 6: Configuración del conjunto de datos real.

7.2 División y normalización

El manifiesto conserva una división estratificada con semilla 42 y sin intersecciones entre subconjuntos. Los parámetros de normalización se calcularon solamente sobre entrenamiento.

Subconjunto	Total	Por clase	Proporción
Entrenamiento	72	24	60 %
Validación	24	8	20 %
Prueba	24	8	20 %

Tabla 7: División estratificada del conjunto real.

Para el canal c se aplica

$$\hat{x}_{n,c} = \frac{x_{n,c} - \mu_c}{\sigma_c}. \quad (5)$$

La Tabla 8 presenta los valores obtenidos durante el entrenamiento del modelo real, con dos decimales para facilitar su lectura.

Canal	Media μ_c	Desviación σ_c
ax	3142,51	5776,78
ay	-571,77	4798,74
az	20304,79	3201,26
gx	104,28	1553,92
gy	-578,39	1474,63
gz	350,47	3027,56

Tabla 8: Parámetros de normalización calculados con el subconjunto de entrenamiento.

7.3 Arquitectura de la CNN

Las convoluciones recorren el eje temporal y combinan los seis canales para detectar patrones locales. La salida Softmax contiene tres valores no negativos cuya suma es uno; la clase se obtiene mediante el índice de mayor probabilidad. La definición de las capas corresponde a las operaciones documentadas por Keras [6].

Capa	Configuración	Salida	Parámetros
Entrada	100 pasos, 6 canales	100×6	0
Conv1D	16 filtros, $k = 5$, valid, ReLU	96×16	496
MaxPooling1D	Tamaño 2	48×16	0
Conv1D	32 filtros, $k = 3$, valid, ReLU	46×32	1568
GlobalAveragePooling1D	Promedio temporal	32	0
Dense	16 unidades, ReLU	16	528
Dense	3 unidades, Softmax	3	51
Total			2643

Tabla 9: Arquitectura y número de parámetros de la CNN 1D real.

7.4 Entrenamiento y evaluación

El historial contiene 58 épocas y registra exactitud y pérdida para entrenamiento y validación. La mejor época según el criterio de selección fue la 33. Las exactitudes de entrenamiento y validación de la Tabla 10 corresponden a ese *checkpoint*, no al último punto de las curvas. La Figura 6 presenta el historial completo generado a partir de los datos de entrenamiento.

La matriz de la Figura 7 utiliza filas para la clase real y columnas para la predicción. *left* obtuvo 8/8; *right* y *up* obtuvieron 6/8 y se confundieron dos veces en cada sentido. Debido a que el conjunto de prueba tiene solo 24 ejemplos, 83,33 % debe leerse como un resultado preliminar y no como una estimación precisa para usuarios o condiciones no observadas.

Evolución del entrenamiento del modelo real_v1

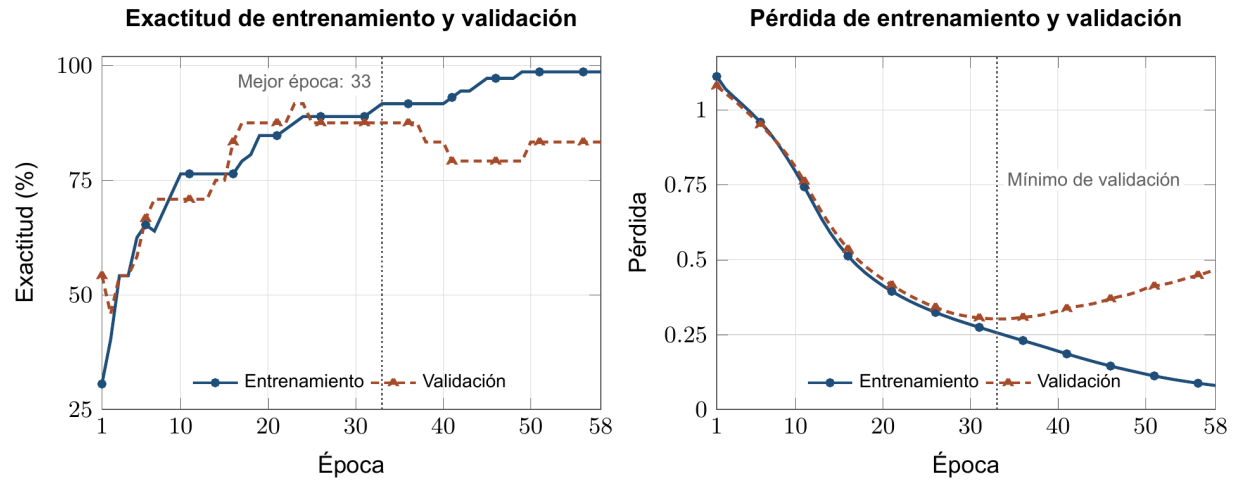


Figura 6: Historial completo de exactitud y pérdida durante 58 épocas; la línea vertical identifica el *checkpoint* seleccionado en la época 33.

Métrica	Resultado
Exactitud de entrenamiento, época 33	91,67 %
Exactitud de validación, época 33	87,50 %
Pérdida de prueba	0,34613
Exactitud de prueba	83,33 %
Aciertos de prueba	20 de 24
Épocas ejecutadas	58
Checkpoint seleccionado	33

Tabla 10: Métricas del *checkpoint* seleccionado y del conjunto de prueba.

Matriz de confusión del conjunto de prueba

Exactitud global: 83,33 % (20/24 predicciones correctas)

Clase real	left	8	0	0
	right	0	6	2
	up	0	2	6
		left	right	up
		Clase predicha		

Figura 7: Matriz de confusión del conjunto de prueba: 20 aciertos en 24 capturas.

7.5 Exportación e inferencia NumPy

El paquete real conserva pesos NumPy, etiquetas, normalización, configuración, métricas, historial, división y manifiesto. Los hashes declarados por el manifiesto resultaron coherentes. La reconstrucción en el cuaderno implementa convoluciones con relleno *valid*, agrupamiento, promedio global, capas densas, ReLU y Softmax mediante NumPy [7]. Al comparar sus salidas con Keras, la diferencia absoluta máxima fue $1,788 \times 10^{-7}$ y las clases predichas coincidieron.

No obstante, el archivo `AI/gesture_model_real_v1.zip` no contiene `numpy_gesture_inference.py`. El módulo disponible en el paquete sintético utiliza convolución *same*, nombres de pesos distintos y una estructura de metadatos incompatible. Por tanto, la equivalencia algorítmica está verificada fuera de línea, pero el paquete local real no basta por sí solo para reconstruir el ejecutable desplegado en la Raspberry. Esta diferencia puede explicar el desacople entre las métricas del modelo real y el resultado de una prueba final.

8 Software de la Raspberry Pi e interfaz IoT

8.1 Servidor Flask

Flask permite exponer rutas HTTP y respuestas JSON con una aplicación Python compacta [5]. La interfaz prevista del proyecto se resume en la Tabla 11.

Durante el desarrollo, la aplicación Flask se ejecutó en la Raspberry Pi y se comprobaron los endpoints `/api/status`, `/api/dataset` y `/api/gesture`, incluidas solicitudes exitosas de captura

Método	Ruta	Función
GET	/	Presentar el dashboard web.
GET	/api/status	Entregar el último estado, resultado y tiempo de actualización.
POST	/api/dataset	Recibir y almacenar una captura con etiqueta durante la fase de adquisición.
POST	/api/gesture	Validar 100 muestras, normalizar, inferir y devolver clase, confianza y probabilidades.

Tabla 11: Contrato de endpoints utilizado durante el desarrollo.

e inferencia. La copia local `RASPBERRY/server/app.py` corresponde a un servidor de demostración que devuelve una clase simulada con 0,99 de confianza, mientras `RASPBERRY/app_before_dashboard_v1.py` contiene una ruta de inferencia orientada al modelo real. Los respaldos incluyen otras etapas del servicio, pero no permiten determinar qué combinación exacta de aplicación y modelo permaneció activa en la última prueba.

8.2 Prueba del dashboard y el monitor serial

La Figura 8 reúne el dashboard y el monitor serial durante una transacción completa. La prueba utilizó 100 muestras, una duración de captura de aproximadamente 1981,7 ms y obtuvo una respuesta HTTP 200 con clase `left`, confianza cercana a 60,75 % y tiempo de inferencia de 117,621 ms mediante `numpy_cnn_1d`. Esta ejecución corresponde a una etapa anterior del cliente, cuando el APDS aún se conservaba como referencia, y no registró el hash del modelo ejecutado.

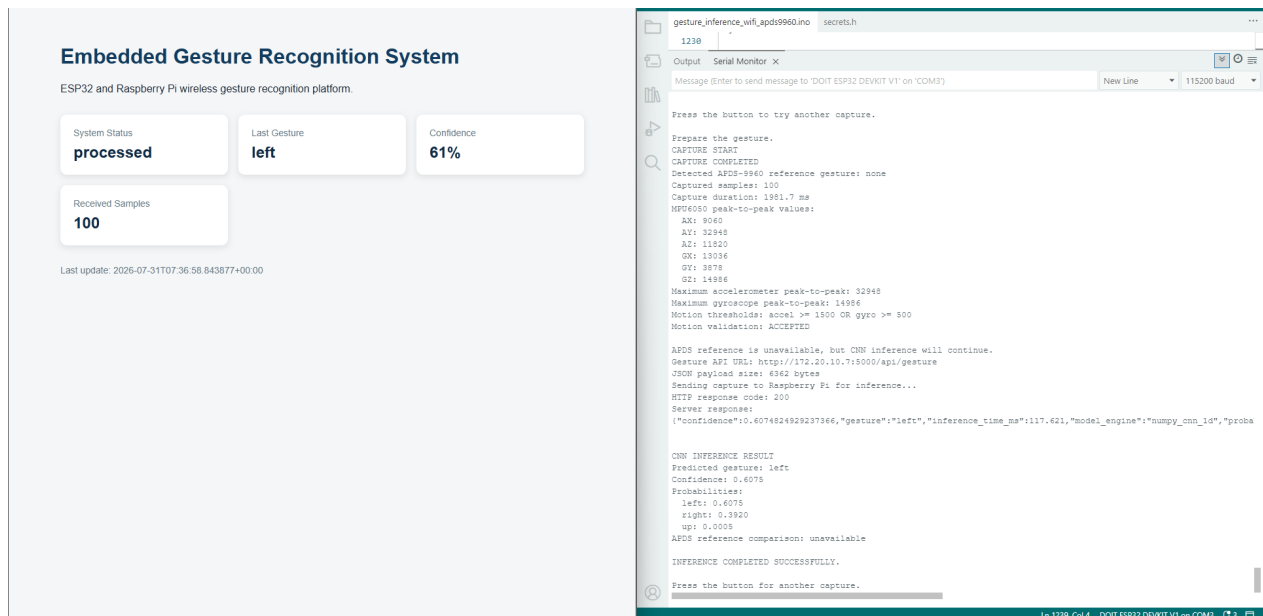


Figura 8: Dashboard Flask y monitor serial durante una solicitud de 100 muestras con respuesta `left`. La ejecución corresponde a una etapa anterior del cliente.



8.3 Arranque, red y seguridad

El servicio `gesture-server.service` se configuró para ejecutar la aplicación bajo el usuario `saló`, utilizar el entorno virtual del proyecto y reiniciarse ante fallos. Durante las pruebas se comprobó que el servicio estaba activo y que Flask respondía en los endpoints previstos. Para la red de demostración se configuró la dirección fija 172.20.10.7. Aunque no se conservó una captura final de `systemctl status` o `journalctl`, el funcionamiento del servidor sí fue verificado; la incertidumbre pendiente corresponde a la versión exacta del modelo utilizada en la última ejecución.

El transporte implementado usa HTTP sin cifrado y el contrato no incluye autenticación. Las credenciales Wi-Fi se mantienen en archivos `secrets.h` y no se reproducen en este informe. Para un despliegue distinto de la demostración local se requerirían control de acceso, protección de credenciales, límites de tamaño y tiempo, y validación estricta de tipos y rangos.

9 Diseño de la PCB

9.1 Alcance del diseño

Como actividad de integración se elaboró una tarjeta en KiCad 9.0.7. El diseño reúne el ESP32-WROOM-32D, USB-C, regulador AMS1117-3.3, CH340C, circuito de auto-programación y conectores para MPU6050, APDS-9960, OLED, servo, botón, LED y buzzer. La Figura 9 muestra una exportación del esquemático. En esta etapa no se generaron Gerbers ni una lista de materiales definitiva, y tampoco se completaron los reportes ERC/DRC, el ensamblaje o las pruebas eléctricas.

La vista de la Figura 10 muestra cuatro perforaciones de montaje, la colocación de los componentes, el ruteo y una zona de exclusión de 5 mm frente a la antena. La PCB no fue fabricada, por lo que su comportamiento eléctrico quedó fuera del alcance de esta entrega.

9.2 Correspondencia y revisión previa a fabricación

MPU6050, OLED, servo, botón y LED coinciden con los pines de v4. El buzzer aparece conectado a GPIO25 mediante un 2N2222A, pero v4 no lo controla. Además, los valores del esquema, 4,7 k Ω en base y 47 k Ω como polarización, difieren de los valores aproximados de 1 k Ω y 10 k Ω documentados para el prototipo independiente.

Durante la revisión previa a fabricación se identificaron las discrepancias de la Tabla 12. Los puntos relativos al CH340C y al auto-programado deben comprobarse contra el símbolo y la huella definitivos; la hoja de datos del fabricante define las condiciones de V3 [8] y las referencias de Espressif permiten contrastar la interfaz de programación y el diseño alrededor de la antena [9].

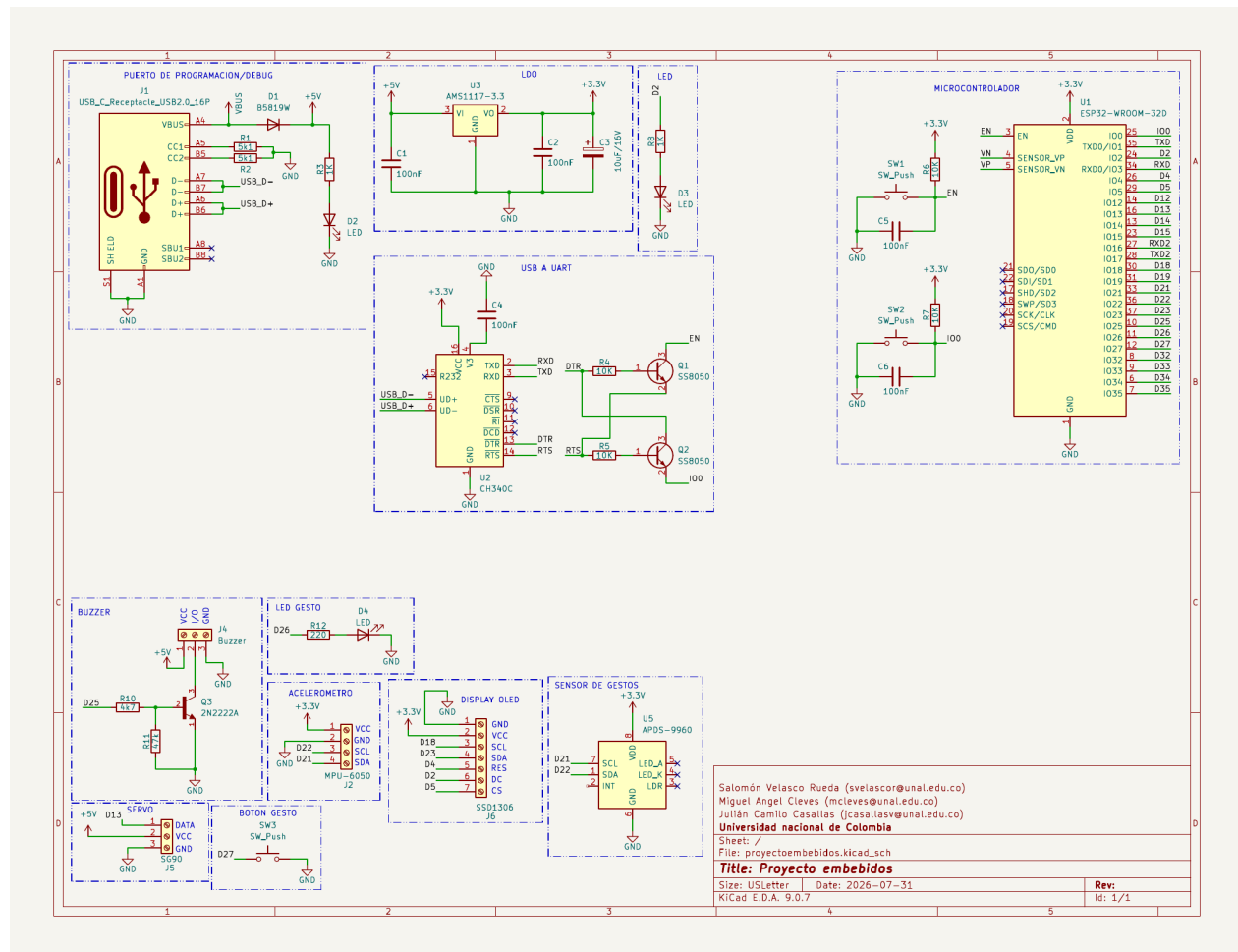


Figura 9: Esquemático de la tarjeta propuesta, con alimentación USB-C, interfaz USB-UART, ESP32-WROOM-32D y conexiones para sensores y actuadores. El diseño se realizó en KiCad y no alcanzó la etapa de fabricación o validación eléctrica.

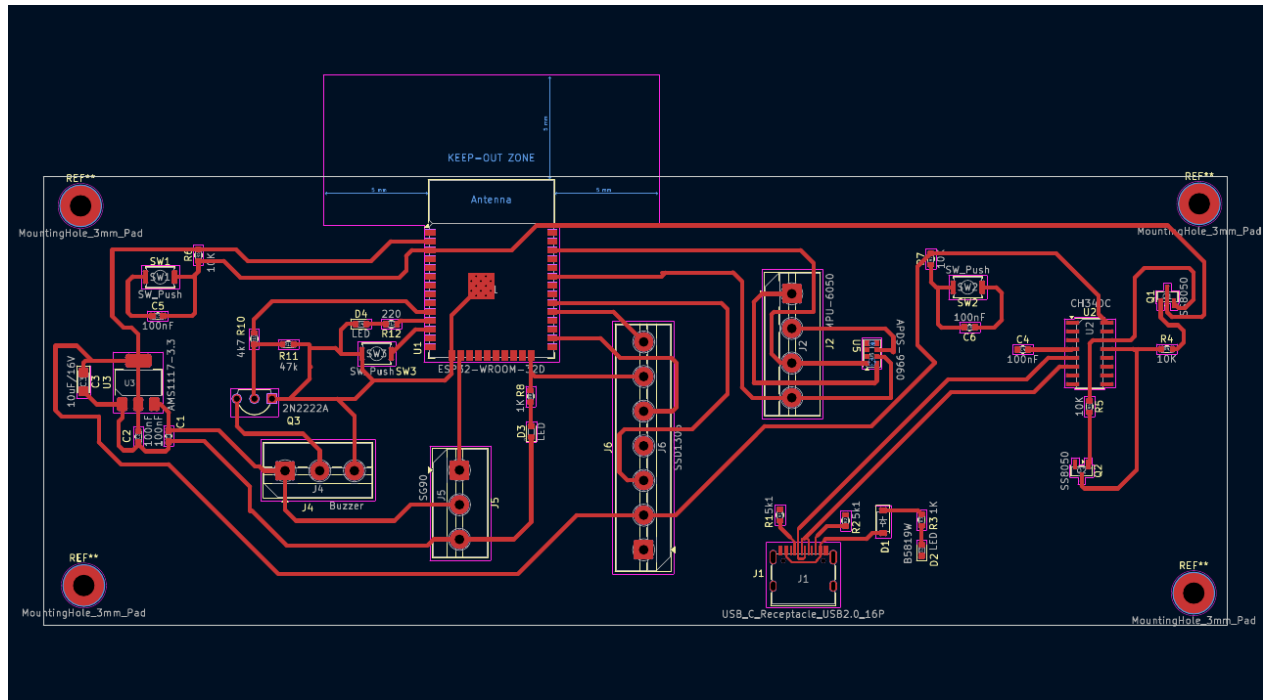


Figura 10: Vista de colocación y ruteo de la PCB propuesta, incluyendo el módulo ESP32, conectores de periféricos y zona de exclusión de la antena. El diseño no fue fabricado ni validado mediante pruebas eléctricas.

Hallazgo	Certeza	Acción antes de fabricar
USB_D- llega a UD+ y USB_D+ a UD- del CH340C.	Alta	Corregir el cruce; de lo contrario puede impedir la enumeración USB.
CH340C con VCC a 3,3 V, mientras V3 solo tiene un capacitor a tierra.	Alta sobre el dibujo	Revisar el modo de 3,3 V y unir V3 según la hoja de datos aplicable.
APDS: SCL en GPIO21 y SDA en GPIO22.	Alta	Intercambiar SDA y SCL para coincidir con Wire.begin(21, 22) y con el MPU6050.
Q2 del auto-programado parece invertir colector y emisor respecto a la referencia.	Media-alta	Verificar numeración del símbolo, huella y orientación física.
GPIO2 comparte OLED-DC con un LED y su resistencia.	Alta	Revisar la carga sobre una señal de control y sobre un pin de arranque.
Entradas VP/VN aparecen intercambiadas en el conector.	Alta	Corregir nombres o redes, aunque v4 no las use.
No se muestran pull-ups I2C ni reserva local cerca del servo.	Alta / media	Confirmar resistencias en módulos y dimensionar desacople y retorno de potencia.
Referencias REF**, serigrafías superpuestas y ausencia de ERC/DRC.	Alta	Anotar, limpiar y ejecutar las verificaciones antes de generar Gerbers.

Tabla 12: Hallazgos del esquemático y la vista de PCB que impiden considerarlos listos para fabricación.



10 Resultados y discusión

10.1 Resultados principales

Los principales resultados cuantitativos fueron: 120 capturas balanceadas y estructuralmente válidas; división 72/24/24; arquitectura de 2643 parámetros; historial de 58 épocas; prueba de 20/24; matriz $[[8, 0, 0], [0, 6, 2], [0, 2, 6]]$; y equivalencia Keras-NumPy con diferencia máxima de $1,788 \times 10^{-7}$. Estos valores corresponden al modelo real evaluado fuera de línea; en las pruebas finales no se registró un identificador que permitiera confirmar automáticamente la versión activa.

Las pruebas funcionales incluyeron el prototipo energizado, la integración del ESP32, la OLED y el servo, una pantalla con resultado RIGHT al 49,42 % y una transacción con respuesta left al 60,75 %. No se realizó una campaña sincronizada que registrara para cada gesto la entrada, la predicción y el desplazamiento del servo; por ello no se calculó una exactitud en vivo ni una tasa de respuesta mecánica para las tres clases.

10.2 Fallo de cierre: *up* clasificado como *left*

En una de las pruebas finales, el MPU6050 produjo una captura válida, pero el sistema respondió *left* ante un gesto *up*. Este comportamiento no coincide con la matriz del modelo real, en la que *up* solo se confundió con *right*. La configuración del modelo real utiliza el orden de clases *left*, *right*, *up* y de canales *ax*, *ay*, *az*, *gx*, *gy*, *gz*; sin embargo, en esa ejecución no quedó registrado el identificador del paquete cargado por el servicio.

Las posibles causas son: uso del paquete sintético anterior; etiquetas, canales o normalización diferentes en el proceso activo; orientación física distinta del MPU6050; una transformación adicional en otra versión del cliente; o ejecución del gesto fuera del patrón aprendido. Como esa prueba no quedó asociada al JSON de entrada, al hash del paquete y a la respuesta detallada, no es posible atribuir la falla únicamente a la CNN, al firmware o al hardware.



10.3 Problemas encontrados y medidas implementadas

Problema	Medida implementada	Estado
ID APDS-9960 0xA8 no admitido	Ampliación de la comprobación de identidad en la biblioteca SparkFun.	Cambio implementado; sensor no usado por v4.
Doble captura por pulsador	Deshabilitación del evento durante el ciclo y validación de pulsación/liberación estable.	Implementado en v2-v4; sin campaña cuantitativa de rebote.
Fallos I2C del MPU6050	WHO_AM_I, tiempo límite, pulsos SCL, STOP y reintentos.	Implementado y reforzado en v3/v4.
Lecturas nulas o congeladas	Prueba de magnitud mínima y detección de cinco muestras idénticas.	Implementado en v4; cubre patrones específicos.
Capturas sin movimiento	Rangos pico a pico y criterio AND de 2500/2500.	Implementado; los 120 archivos finales superan el criterio definido.
Ruido o carga del servo	Adjuntar, mantener 300 ms y desacoplar la señal.	Implementado en v3/v4; falta medición eléctrica.
Runtime pesado en ARMv6	Reconstrucción manual de capas con NumPy.	Equivalencia fuera de línea verificada; empaquetado real local incompleto.
Resultado final incoherente	Revisión de etiquetas, canales, normalización y versiones.	Causa no resuelta; requiere trazabilidad en ejecución.

Tabla 13: Problemas del desarrollo, mitigaciones y nivel de cierre.

10.4 Estado final del proyecto

Subsistema	Estado final	Interpretación
Prototipo sobre protoboard	Integrado físicamente	Montaje energizado con sensores, interfaz y actuadores conectados.
Firmware v4	Implementado y probado	Incluye las protecciones descritas; queda pendiente una campaña completa de regresión.
Dataset real	Verificado	120/120 archivos válidos, 40 por clase y forma uniforme.
CNN real	Evaluada fuera de línea	83,33 % sobre 24 ejemplos; principal confusión $right \leftrightarrow up$.
Inferencia NumPy	Verificada en cuaderno	Salida equivalente a Keras; módulo desplegable no incluido en el ZIP real local.
Servidor y dashboard	Funcionamiento comprobado	El servicio y los endpoints respondieron correctamente; no se registró la versión exacta del modelo en la última prueba.
OLED, servo y LED	Integrados en v4	La OLED mostró resultados; falta una prueba sincronizada de actuación para las tres clases.
Buzzer	Prueba separada / PCB	No forma parte de v4.
PCB	Diseño preliminar	No fabricada y con correcciones obligatorias.

Tabla 14: Estado final de los subsistemas al cierre del informe.

11 Limitaciones y trabajo pendiente

Las principales limitaciones experimentales son el tamaño reducido del conjunto de prueba, la recolección bajo condiciones y usuarios limitados, la ausencia de mediciones de latencia y frecuencia sobre una campaña controlada, y la falta de una prueba final reproducible que relacione gesto, entrada cruda, versión del modelo, salida, pantalla y actuador. Una sola confianza alta o baja no reemplaza la evaluación por clase ni detecta un despliegue equivocado.

El trabajo pendiente prioritario es:

1. Construir un paquete inmutable del modelo real que incluya el módulo NumPy compatible, pesos, normalización, etiquetas, orden de canales y manifiesto; exponer en `/api/status` la versión y el hash activos.
2. Repetir pruebas controladas para las tres clases, guardando el JSON enviado, orientación del sensor, respuesta completa, tiempo, resultado en OLED y posición del servo. Deben incluirse un tiempo límite para la liberación del pulsador y un umbral de confianza o clase de rechazo antes de actuar.
3. Aumentar el dataset con varios usuarios, velocidades, orientaciones y sesiones; emplear validación cruzada o un conjunto externo para estimar mejor la generalización.



4. Medir el intervalo real entre muestras, la latencia HTTP y el tiempo de extremo a extremo, y registrar reconexiones de Wi-Fi y recuperación del bus.
5. Corregir USB D+/D-, V3 del CH340C, APDS SDA/SCL, auto-programado, VP/VN, GPIO2, polarización I2C, alimentación del servo y serigrafía; después ejecutar ERC/DRC y generar Gerbers revisados.
6. Si se incorpora el buzzer, definir su función en el protocolo de estados, comprobar los valores del transistor y verificar que la alimentación no perturbe la adquisición.
7. Añadir autenticación, límites de carga, protección de credenciales y transporte seguro cuando el sistema se utilice fuera de una red de demostración controlada.

12 Conclusiones

El proyecto alcanzó una integración física y de software significativa: se construyó un prototipo con ESP32, MPU6050, OLED, servo, LED y comunicación Wi-Fi, y se desarrolló una aplicación de servidor orientada a recibir capturas e inferir gestos. La separación entre adquisición y procesamiento fue adecuada para una plataforma embebida distribuida y permitió iterar el modelo sin trasladar toda la CNN al microcontrolador.

El conjunto de datos real quedó bien definido en su alcance: 120 secuencias balanceadas de 100×6 , con una división estratificada y normalización calculada solo en entrenamiento. La CNN compacta de 2643 parámetros obtuvo 83,33 % en un conjunto de prueba de 24 ejemplos. Este resultado demuestra aprendizaje sobre los datos recopilados, pero su incertidumbre y la confusión *right*↔*up* impiden generalizarlo como desempeño definitivo.

La versión v4 incorpora mejoras concretas frente a fallos observados: menor frecuencia I2C, recuperación del bus, rechazo de lecturas nulas o congeladas, criterio de movimiento más restrictivo y control reforzado del pulsador. El servidor Flask, los endpoints y la inferencia NumPy funcionaron durante las pruebas, y la OLED presentó los resultados recibidos. Aun así, quedó pendiente una campaña sincronizada que evaluara las tres clases junto con la respuesta de los actuadores.

La diferencia entre el paquete real evaluado y los archivos disponibles para el despliegue es el principal riesgo de trazabilidad. El fallo *up*→*left* no coincide con la matriz del modelo real evaluado y permanece abierto. La solución debe empezar por identificar el modelo activo mediante hash, reunir el módulo NumPy compatible y registrar la misma entrada en cada etapa antes de modificar nuevamente el entrenamiento o el firmware.

Finalmente, la PCB constituye un entregable de diseño y no un prototipo validado. Las coincidencias de pines en varios periféricos son útiles, pero las discrepancias de USB, CH340C y APDS, junto con la ausencia de ERC/DRC y fabricación, obligan a corregirla y revisarla antes de producirla. El diseño realizado establece una base para una futura versión integrada del sistema.

A Trazabilidad de los archivos principales

Ruta relativa	Uso en el informe
ESP32/gesture_inference_wifi_oled_servo_v2/gesture_inference_wifi_oled_servo_v2.ino	Línea base coherente para la comparación de versiones.
ESP32/gesture_inference_wifi_oled_servo_v3/gesture_inference_wifi_oled_servo_v3.ino	Recuperación I2C, APDS deshabilitado y servo desacoplado.
ESP32/gesture_emergency_stable_v4/gesture_emergency_stable_v4.ino	Fuente principal para el comportamiento final documentado.
DATASET/real_dataset_120_v1/dataset/raw	Los 120 JSON usados para distribución y validaciones estructurales.
DATASET/real_dataset_120_v1/dataset_config.json	Configuración del conjunto real.
DATASET/GESTURE_CAPTURE_PROTOCOL.md	Definición de la ejecución de los gestos.
AI/gesture_model_real_v1.zip	Pesos, configuración, etiquetas, normalización, métricas, historial, división y manifiesto del modelo real.
AI/gesture_cnn_real_dataset_pipeline_v1.ipynb	Entrenamiento y comprobación Keras-NumPy.
AI/gesture_model_numpy_inference_v1/numpy_gesture_inference.py	Módulo del modelo sintético; se documenta su incompatibilidad con el paquete real.
RASPBERRY/server/app.py	Servidor local de demostración, identificado como simulación.
RASPBERRY/app_before_dashboard_v1.py	Ruta anterior de inferencia.
BACKUPS/flask_numpy_inference_working_v1	Variantes del servidor y unidad de servicio.
BACKUPS/apds9960_gesture_working_v1/PATCHED_LIBRARY	Biblioteca APDS adaptada para el identificador 0xA8.
DOCUMENTACIÓN/INFORME/IMAGENES/GENERADAS/generar_graficas.py	Reproducción de distribución, curvas y matriz desde los datos reales.

Tabla 15: Trazabilidad entre afirmaciones técnicas y archivos del proyecto.

Referencias

- [1] Espressif Systems, *ESP32-WROOM-32 Datasheet*, versión 3.6. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf. Consultado el 1 de agosto de 2026.
- [2] Invensense, *MPU-6000 and MPU-6050 Product Specification*, revisión 3.4. Disponible en: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet.pdf>. Consultado el 1 de agosto de 2026.
- [3] Broadcom, *APDS-9960 Digital RGB, Ambient Light, Proximity and Gesture Sensor*. Disponible en: <https://www.broadcom.com/products/optical-sensors/integrated-ambient-light-and-proximity-sensors/apds-9960>. Consultado el 1 de agosto de 2026.



- [4] Raspberry Pi Ltd., *Raspberry Pi computer hardware documentation*. Disponible en: <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>. Consultado el 1 de agosto de 2026.
- [5] Pallets Projects, *Flask Documentation*. Disponible en: <https://flask.palletsprojects.com/en/stable/>. Consultado el 1 de agosto de 2026.
- [6] Keras, *Keras Layers API*. Disponible en: <https://keras.io/api/layers/>. Consultado el 1 de agosto de 2026.
- [7] NumPy Developers, *NumPy Reference*. Disponible en: <https://numpy.org/doc/stable/>. Consultado el 1 de agosto de 2026.
- [8] Nanjing Qinsheng Microelectronics, *CH340 Datasheet*. Disponible en: <https://www.wch-ic.com/downloads/CH340DS1.PDF.html?type=en>. Consultado el 1 de agosto de 2026.
- [9] Espressif Systems, *ESP32 Hardware Design Guidelines*. Disponible en: <https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32/>. Consultado el 1 de agosto de 2026.